

3.1. DDL (Data Definition Language)

El DDL (Data Definition Language) es quien se encarga de definir las Bases de Datos como así también los elementos de nuestra BD: tablas, vistas e índices. Admite las siguientes sentencias de definición:

- **CREATE:** se utiliza esta sentencia para crear bases de datos, tablas, dominios, aserciones y vistas.
- **ALTER:** esta sentencia nos permite modificar tablas y dominios.
- **DROP:** se emplea para borrar bases de datos, tablas, dominios, aserciones y vistas.

Cada una de estas sentencias se puede aplicar a las tablas, vistas, procedimientos almacenados y triggers de la base de datos.

De este modo, podemos:

- Añadir una nueva base de datos.
- Suprimir una base de datos.
- Añadir una nueva tabla a la base de datos.
- Suprimir una tabla de la base de datos.
- Modificar la estructura de una tabla existente.
- Añadir una nueva vista a la base de datos.
- Suprimir una vista de la base de datos.
- Construir un índice para una columna.
- Suprimir el índice para una columna.
- Definir un alias para un nombre de tabla.
- Suprime un alias para un nombre de tabla.

Lo primero que haremos para poder trabajar con bases de datos relacionales es definirlos.

1. Crear una base de datos

Desde el punto de vista de SQL, una base de datos es sólo un conjunto de relaciones (o tablas). Para organizarlas o visualizarlas se accede a ellas mediante su nombre. A nivel de sistema operativo, cada base de datos se guarda en un directorio diferente.

Cuando queramos crear una base de datos, no tendremos ninguna tabla, estará vacía nuestra BD, pero es una tarea muy simple agregar una tabla..

Empezaremos creando nuestra propia base de datos y manipulándola, mientras nos adentramos en el funcionamiento de MySQL

CREATE

Cuando utilicemos este comando se creará un objeto dentro de la base de datos. Dependiendo de lo que queramos crear, se colocará una palabra clave, Esto puede ser la propia base de datos, una tabla, vista, índice, trigger, función, procedimiento o cualquier otro objeto que el motor de la base de datos soporte. Empezaremos creando una base de datos utilizando la sentencia **CREATE DATABASE**.

DATABASE:

```
CREATE DATABASE nombre_de_la_base;
```

Se puede completar la orden con la cláusula:

```
CREATE DATABASE IF NOT EXISTS nombre_de_la_base;
```

Si no usamos IF NOT EXISTS y ya existe una base con ese nombre, MySQL nos avisará del error y no ejecutará acción alguna. Ya que la base de datos no podrá crearse si previamente existe una con el mismo nombre

Una vez que la base está creada, podemos seleccionarla como BD por defecto. De esta manera no tendremos que colocar el nombre de nuestra base de datos cada vez que necesitamos trabajar con ella. Para seleccionar una base de datos se usa el comando USE, que no es exactamente una sentencia SQL, sino más bien una opción de MySQL.

```
USE nombre_de_la_base;
```

2. Crear una tabla

Ahora vamos a crear una tabla utilizando la sentencia: CREATE TABLE. Al momento de utilizarla poseemos diferentes características para crear nuestra tabla. La sentencia CREATE TABLE creará una tabla con las columnas que nosotros le indiquemos.

Por ejemplo, crearemos una tabla que almacena nombres de personas y la fecha de nacimiento de cada uno. Colocaremos el nombre de la tabla y los nombres y tipos de las columnas.

```
CREATE TABLE nombre_de_tabla (nombre_del_campo tipo de campo, ...);
```

Por ejemplo:

```
CREATE TABLE personas (nombre VARCHAR(50), fecha DATE);
```

Utilizando esta sentencia hemos creado una nueva tabla llamada "personas" con dos columnas: una de ellas es "nombre" que puede contener cadenas de hasta 50 caracteres y la otra columna es "fecha" de tipo fecha.

Podemos realizar consultas, mostrando cuántas tablas hay y qué nombres tienen en una base de datos, usando la sentencia SHOW TABLES:

```
SHOW TABLES;
```

Como indicamos anteriormente, a la hora de definir los campos de nuestra tabla tendremos muchas más opciones. Además del tipo y el nombre, podemos definir valores por defecto, permitir o no que contengan valores nulos, crear una clave primaria, indexar...

La sintaxis para definir campos es:

```
nombre_col tipo [NOT NULL | NULL] [DEFAULT valor_por_defecto] [AUTO_INCREMENT] [[PRIMARY] KEY]  
[COMMENT 'string']
```

```
[definición_referencia]
```

Veamos cada una de las opciones por separado detallando cada una.

1. Valores nulos

El concepto del valor NULL muchas veces confunde a los recién llegados a SQL, ya que frecuentemente creen que NULL es lo mismo que una cadena de caracteres vacía ' ' o el valor cero, pero no lo es.

Los valores NULL indican que los datos son desconocidos, no aplicables o que se agregarán más adelante. Por ejemplo, la inicial de un cliente puede que no sea conocida en el momento en que éste hace un pedido.

No hay dos valores NULL iguales. Es por esto que la comparación entre dos valores NULL, o entre un valor NULL y cualquier otro valor, tiene un resultado desconocido.

A la hora de definir una nueva columna podremos decidir si podrá contener valores nulos. Debemos tener en cuenta que, las columnas que son o forman parte de una clave primaria no pueden tener valores nulos. Al definir una columna como clave primaria, automáticamente se impide que pueda contener valores nulos.

La opción por defecto permite valores nulos, NULL. Para restringirlo se deberá usar NOT NULL para que no se permitan dichos valores nulos.

Por ejemplo:

```
CREATE TABLE provincia1 (nombre CHAR(20) NOT NULL, poblacion INT NULL);
```

2. Valores por defecto

Cada columna puede poseer un valor por defecto que se asignará de forma automática cuando no se especifique un valor determinado al añadir filas en la tabla. Se utilizará DEFAULT.

Si una columna puede tener un valor nulo, y no se especifica un valor por defecto, se usará NULL como valor por defecto. En el ejemplo anterior, el valor por defecto para población es NULL

Por ejemplo, si queremos que el valor por defecto para población sea 5000, podemos crear la tabla como:

```
CREATE TABLE provincia2 (nombre CHAR(20) NOT NULL, poblacion INT NULL DEFAULT 5000);
```

3. Claves primarias

También se puede definir una clave primaria sobre una columna, usando la palabra clave KEY o PRIMARY KEY. Sólo puede existir una clave primaria en cada tabla, y la columna sobre la que se define una clave primaria no puede tener valores NULL. Si esto no se especifica de forma explícita, MySQL lo hará de forma automática.

Por ejemplo, si queremos crear un índice en la columna nombre de la tabla de provincias, crearemos la tabla así:

```
CREATE TABLE provincia3 (nombre CHAR(20) NOT NULL PRIMARY KEY, poblacion INT NULL DEFAULT 5000);
```

Usar NOT NULL PRIMARY KEY equivale a PRIMARY KEY, NOT NULL KEY o sencillamente KEY.

4. Campos autoincrementales

Podremos también crear un campo que se incremente automáticamente usando AUTO_INCREMENT. Esta columna sólo podrá contener datos de tipo enteros. Este campo hará que si a la hora de agregar una fila se omite el valor de la columna autoincrementada o si se inserta un valor nulo, su valor se colocará automáticamente, tomando el valor más alto que hay y sumándole una unidad. Esto permite crear, de una forma sencilla, una columna con un valor único para cada fila de la tabla.

Las columnas con campos autoincrementales se suelen utilizar como claves primarias 'artificiales'.

```
CREATE TABLE provincia4 (clave INT AUTO_INCREMENT PRIMARY KEY, nombre CHAR(20) NOT NULL, poblacion INT NULL DEFAULT 5000);
```

5. Comentarios

Cuando creamos la tabla, podemos añadir un comentario a cada columna como información adicional sobre alguna característica especial de la columna, utilizando COMMENT y entra en el apartado de documentación de la base de datos:

```
CREATE TABLE provincia5 (clave INT AUTO_INCREMENT PRIMARY KEY COMMENT 'Clave principal', nombre CHAR(50) NOT NULL, poblacion INT NULL DEFAULT 5000);
```

3.2. Índice

Un índice (o KEY, o INDEX) es un grupo de datos que MySQL asociado con una o varias columnas de la tabla. En este grupo de datos aparece la relación entre el contenido y el número de fila donde está ubicado. Los índices, como los índices de los libros, sirven para agilizar y optimizar las consultas a las tablas, evitando que MySQL tenga que revisar todos los datos disponibles para devolver el resultado. Los índices se utilizan para buscar las filas con los valores de una columna específica rápidamente.

Si una tabla tiene 1000 filas por ejemplo, utilizando INDEX es por lo menos 100 veces más rápido que realizando una lectura secuencial.

Si la tabla tiene un índice para las columnas, MySQL puede determinar rápidamente cuál es la posición del dato buscado sin tener que revisar toda la tabla.

Podemos crear el índice a la vez que creamos la tabla, usando la palabra INDEX seguida del nombre del índice a crear y columnas a indexar (que pueden ser varias):

```
INDEX nombre_indice (columna_indexada, columna_indexada2...)
```

La sintaxis es ligeramente distinta según la clase de índice:

```
PRIMARY KEY (nombre_columna_1 [,nombre_columna2...])
```

```
UNIQUE INDEX nombre_indice (columna_indexada1 [,columna_indexada2 ...])
```

```
INDEX nombre_index (columna_indexada1 [,columna_indexada2...])
```

Vamos a distinguir tres tipos de índices:

1. El primero tipo de índice corresponde a las claves primarias, o PRIMARY KEY, que como vimos, también se pueden crear en la parte de definición de columnas.
2. El segundo tipo de índice permite definir índices sobre una columna, sobre varias, o sobre partes de columnas. Para definirlos empleamos KEY o INDEX.
3. El tercero nos permite definir índices con claves únicas, también sobre una columna, sobre varias o sobre partes de columnas. Para definir índices con claves únicas se usa la opción UNIQUE.

A continuación los detallamos.

1. Claves primarias: PRIMARY KEY

La sintaxis para definir claves primarias es:

```
definición_columnas... PRIMARY KEY (index_nombre_col,...)
```

El ejemplo anterior que vimos para crear claves primarias, usando esta sintaxis, quedaría así:

```
CREATE TABLE provincia6 (nombre CHAR(20) NOT NULL, poblacion INT NULL DEFAULT 5000, PRIMARY KEY (nombre));
```

A su vez, podemos crear varias claves primarias en la misma sentencia colocándolas entre paréntesis, de la siguiente manera:

```
CREATE TABLE mitabla1 (id1 CHAR(2) NOT NULL, id2 CHAR(2) NOT NULL, texto CHAR(30), PRIMARY KEY (id1, id2));
```

2. Índices: KEY o INDEX

Para definir estos índices se usan indistintamente las opciones KEY o INDEX.

```
CREATE TABLE mitabla2 (id INT, nombre CHAR(19), INDEX (nombre));
```

O su equivalente:

```
CREATE TABLE mitabla3 (id INT, nombre CHAR(19), KEY (nombre));
```

También podemos crear un índice sobre parte de una columna:

Este ejemplo usará sólo los cuatro primeros caracteres de la columna 'nombre' para crear el índice.

```
CREATE TABLE mitabla4 ( id INT, nombre CHAR(19), INDEX (nombre(4)));
```

3. Claves únicas: **UNIQUE**

Para definir índices con claves únicas se usa la opción **UNIQUE**.

Un índice único no permite la inserción de filas con claves repetidas. La excepción es el valor **NULL**, que sí se puede repetir.

```
CREATE TABLE mitabla5 (id INT, nombre CHAR(19), UNIQUE (nombre));
```

Tanto los índices normales como los de claves únicas pueden tomar valores **NULL**. A diferencia de las claves primarias, que no pueden contener valores nulos. Es por esto que las siguientes definiciones son equivalentes:

```
CREATE TABLE mitabla6 (id INT, nombre CHAR(19) NOT NULL, UNIQUE (nombre));
```

Y:

```
CREATE TABLE mitabla7 (id INT, nombre CHAR(19), PRIMARY KEY (nombre));
```

Claves Externas o Foráneas

En MySQL, InnoDB es el único motor de almacenamiento que soporta claves foráneas en sus tablas. Sin embargo, esto no impide usarlas en otros tipos de tablas.

La diferencia consiste en que en esas tablas no se verifica si una clave foránea existe realmente en la tabla referenciada, como si se hace en tablas InnoDB y además que no se eliminan filas de una tabla con la definición de una clave foránea.

Hay dos modos de definir claves foráneas en bases de datos MySQL:

- El primero, sólo sirve para documentar. De esta manera, determinaremos una referencia a la vez que se define una columna:

```
CREATE TABLE personas (  
id INT AUTO_INCREMENT PRIMARY KEY,  
nombre VARCHAR(40),  
fecha DATE);
```

```
CREATE TABLE telefonos (  
  
numero CHAR(12),  
idtel INT NOT NULL REFERENCES personas (id) ON DELETE CASCADE ON UPDATE CASCADE);
```

Hemos definido la referencia 'id' de la tabla 'telefonos', insertando una clave foránea a la columna 'id' en la tabla 'personas'. Pero esta definición no supone ningún comportamiento por parte de MySQL.

- La segunda manera de definir las, es mucho más útil, aunque sólo se aplica a tablas InnoDB. No se añade la referencia en la definición de la columna, se agrega luego de la definición de todas las columnas.

El ejemplo anterior, usando tablas InnoDB y esta definición de claves foráneas quedará así:

```
CREATE TABLE personas2 (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    nombre VARCHAR(40),  
    fecha DATE)  
ENGINE=InnoDB;  
  
CREATE TABLE telefonos2 (  
    numero CHAR(12),  
    idtel INT NOT NULL,  
  
    FOREIGN KEY (idtel) REFERENCES personas2 (id)  
    ON DELETE CASCADE ON UPDATE CASCADE)  
ENGINE=InnoDB;
```

La columna que contiene la definición de clave foránea es necesario que esté indexada. MySQL lo hará de manera implícita. si es que no lo declaramos.

De esta manera se define la clave foránea en la columna 'idtel' de la tabla 'telefonos2' que se relaciona con la columna 'id' de 'personas2'. La definición a su vez fija las tareas a realizar en el caso de que se elimine una fila en la tabla 'personas'.

3.3. DROP

El comando DROP es utilizado para eliminar un objeto de la base de datos. Puede aplicarse a una tabla o un índice, como así también a una vista, función, procedimiento o cualquier otro objeto que la base de datos soporte. Se puede combinar con la sentencia ALTER (que veremos en breve). Ejemplo 1:

```
DROP TABLE TABLA_NOMBRE
```

Ejemplo 2:

```
ALTER TABLE TABLA_NOMBRE (DROP COLUMN CAMPO_NOMBRE1 )
```

ELIMINAR UNA TABLA

Algunas veces es necesario eliminar una tabla, ya sea porque es más sencillo crearla de nuevo que modificarla, o porque ya no es necesaria.

Para eliminar una tabla se usa la sentencia DROP TABLE.

La sintaxis es simple:

```
DROP TABLE tbl_name [, tbl_name] ...
```

Por ejemplo:

```
DROP TABLE provincia6;
```

Se pueden añadir las palabras IF EXISTS para evitar errores si la tabla a eliminar no existe.

```
DROP TABLE [IF EXISTS] tbl_name [, tbl_name] ...
```

En nuestro ejemplo:

```
DROP TABLE provincia6;
```

```
ERROR 1051 (42S02): Unknown table 'provincia6'
```

Para evitar el mensaje de error:

```
DROP TABLE IF EXISTS provincia6;
```

ELIMINAR UNA BASE DE DATOS

De modo parecido, se pueden eliminar bases de datos completas, usando la sentencia DROP_DATABASE.

La sintaxis también es muy simple:

```
DROP DATABASE [IF EXISTS] db_name
```

Hay que tener cuidado, ya que al borrar cualquier base de datos se elimina también cualquier tabla que contenga.

3.4. ALTER

El comando ALTER se utiliza para modificar la estructura de un objeto. Se pueden agregar o quitar campos a una tabla, modificar el tipo de un campo, agregar índices o quitarlos, modificar un trigger, entre otras cosas.

Por Ejemplo: Agregar columna a una tabla

```
ALTER TABLE TABLA_NOMBRE (ADD NUEVO_CAMPO INT UNSIGNED)
```

```
ALTER TABLE Empleados (ADD Salario CURRENCY)
```

(Agrega un campo Salario de tipo Moneda a la tabla Empleados.)

De la misma forma podemos eliminar una columna::

```
ALTER TABLE Empleados DROP Salario
```

(Elimina el campo Salario de la tabla Empleados.)

Resumiendo, "ALTER TABLE" se usa para:

- añadir nuevos campos,
- eliminar campos existentes,
- modificar el tipo de dato de un campo,
- cambiar el nombre de un campo,
- agregar o quitar modificadores como "NULL", "UNSIGNED", "AUTO_INCREMENT",
- agregar o eliminar la clave primaria,
- agregar y eliminar índices,
- renombrar una tabla.

"ALTER TABLE" hace una copia temporal de la tabla original, realiza los cambios en la copia, luego borra la tabla original y renombra la copia.

Realizaremos un ejemplo modificando los campos de una tabla.

Para ello utilizamos la tabla "libros", definida con la siguiente estructura:

- código, INT UNSIGNED AUTO_INCREMENT, CLAVE PRIMARIA,
- titulo, VARCHAR(40) NOT NULL,
- autor, VARCHAR(30),
- editorial, VARCHAR (20),
- precio, DECIMAL(5,2) UNSIGNED.

Agregar campos:

Agregaremos el campo "cantidad", que será de tipo SMALLINT UNSIGNED NOT NULL.

La sentencia será de la siguiente manera: primero colocamos "ALTER TABLE" y luego el nombre de la tabla a modificar. Seguido por "ADD" y el nombre del campo, en este caso "cantidad" con su tipo y los modificadores.

```
ALTER TABLE libros ADD cantidad SMALLINT UNSIGNED NOT NULL;
```

Agreguemos otro campo a la tabla:

ALTER TABLE libros ADD edicion DATE;

- Si agregamos un campo que ya existe anteriormente, nos aparecerá un mensaje de error indicando que el campo ya existe en la tabla y la sentencia no se ejecuta.
- Si no especificamos la posición en donde deseamos agregar el campo, se colocará al final de los campos existentes. La manera de indicar la posición deseada será "AFTER" colocándolo así luego del campo especificado.

ALTER TABLE libros ADD CANTIDAD TINYINT UNSIGNED AFTER autor;

Eliminar campos:

Como dijimos anteriormente, "ALTER TABLE" nos permite modificar la estructura de una tabla, podemos usarla entonces también para eliminar un campo, usaremos "DROP" y el nombre del campo a eliminar. Continuamos con nuestra tabla de ejemplo llamada "libros".

Para eliminar el campo "edicion" tipeamos:

ALTER TABLE libros DROP edicion;

- Si tratamos de borrar un campo que no existe en nuestra tabla nos va a aparecer un mensaje de error y la acción no se ejecutará. A su vez podremos eliminar 2 campos en una misma sentencia de esta manera:

ALTER TABLE libros DROP editorial, DROP cantidad;

- Si eliminamos un campo que es parte de un índice, también se borrará el índice. Hay que tener precaución al eliminar un campo ya que éste puede ser una clave primaria, y si lo eliminamos no nos saldrá ninguna advertencia.
- Si una tabla tiene sólo un campo, este no podrá ser eliminado.
- Si eliminamos un campo clave, la clave también se elimina.

ALTER TABLE libros DROP codigo;

Modificar el tipo de dato:

Con "ALTER TABLE" podremos también modificar el tipo de campo, como así también sus atributos. Seguimos utilizando nuestra tabla "libros", definida con la siguiente estructura:

- código, int unsigned,
- titulo, varchar(30) not null,
- autor, varchar(30),
- editorial, varchar (20),
- precio, decimal(5,2) unsigned,
- cantidad int unsigned.

Queremos modificar el tipo del campo "cantidad", lo definiremos como SMALLINT UNSIGNED, ya que no utilizaremos valores mayores a 50000. Usaremos "ALTER TABLE" luego el nombre de la tabla y "MODIFY" seguido del nombre del campo con su tipo y los modificadores.

ALTER TABLE libros MODIFY cantidad SMALLINT UNSIGNED;

Modificaremos también el campo "titulo" para agregar campos de hasta 60 caracteres y a su vez, no permitir valores nulos, tipeamos:

```
ALTER TABLE libros MODIFY titulo VARCHAR(60) NOT NULL;
```

Debemos tener en cuenta que, si por ejemplo, tenemos un campo de texto de longitud 60 y lo modificamos a 40 caracteres, cambiarán los registros que ya están en la tabla, y si este supera el máximo definido, se cortará y quedará incompleto. Por lo cual hay que estar atentos a los cambios que realizamos en las tablas con datos cargados.

Por esto mismo. si un campo permite valores nulos, con registros ya cargados y se lo modifica a "NOT NULL", se modificarán todos los registros con valor nulo por el valor por defecto según el tipo (cadena vacía si es tipo texto y 0 si es numérico).

Otro caso podría ser, en un campo de tipo DECIMAL (5,2) con un registro "900.00" que lo modificamos a "decimal(4,2)". Se cambiará en su lugar, el valor límite más cercano, que sería en este caso "99.99".

Si intentamos definir "auto_increment" un campo que no es clave primaria, no se realizará la acción y nos mostrará un mensaje de error indicándonos que el campo debe ser clave primaria. Por ejemplo:

```
ALTER TABLE libros MODIFY codigo INT UNSIGNED AUTO_INCREMENT;
```

Agregar o quitar campos y atributos:

"ALTER TABLE" combinado con "MODIFY" permite agregar o quitar campos y atributos de campos. Si modificamos el valor por defecto ("DEFAULT") de un campo usando "MODIFY" vamos a tener que colocar su tipo y todos los modificadores otra vez. Si queremos definir únicamente el valor por defecto, podremos realizarlo de la siguiente manera::

```
ALTER TABLE libros ALTER autor SET DEFAULT 'Varios';
```

Para eliminar el valor por defecto podemos usar DROP::

```
ALTER TABLE libros ALTER autor DROP DEFAULT;
```

Cambiar nombre de los campos:

Con "ALTER TABLE" podremos cambiar el nombre de las columnas de una tabla. Usaremos "ALTER TABLE" seguido del nombre de la tabla y colocaremos "CHANGE" el nombre actual y luego el nombre nuevo con su tipo y los modificadores, ya que además del nombre podremos cambiar el tipo y sus modificadores.

Continuamos con nuestra tabla "libros", definida con la siguiente estructura:

- código, int unsigned auto_increment,
- nombre, varchar(60),
- autor, varchar(30),
- editorial, varchar (20),
- costo, decimal(5,2) unsigned,
- cantidad int unsigned,
- clave primaria: código.

Vamos a cambiar el campo "nombre" por "titulo" y no podrá contener valores nulos:

```
ALTER TABLE libros CHANGE nombre titulo VARCHAR(60) NOT NULL;
```

Cambiaremos también el campo "costo" por "precio":

```
ALTER TABLE libros CHANGE costo precio DECIMAL (5,2);
```

Agregar clave primaria:

Con "ALTER TABLE" podremos agregar una clave primaria a una tabla existente. Usamos "ALTER TABLE" con "ADD PRIMARY KEY" y colocaremos el campo entre paréntesis.

Para agregar una clave primaria a una tabla existente usamos:

```
ALTER TABLE libros ADD PRIMARY KEY (codigo);
```

- Si deseamos que nuestra clave primaria sea autoincrementada, deberemos definir este atributo con "MODIFY" y "AUTO_INCREMENT", una vez que está creada.
No podremos definirla como autoincrementada al mismo tiempo que la creamos. Por esto mismo debemos realizar los pasos en orden, ya que no podremos definir el "AUTO_INCREMENT" y luego convertirla en clave primaria.
- No hay que olvidar que en una tabla puede haber solamente una clave primaria, por lo que si queremos agregar una segunda clave primaria, nos saldrá un mensaje de error.
- A su vez si en un campo hay valores repetidos, no podremos seleccionarlo como clave primaria, nos saldrá un mensaje de error y la operación no se realiza.

Eliminar clave primaria:

Podremos eliminar la clave primaria utilizando "ALTER TABLE" seguido de "DROP PRIMARY KEY", no importa si fue definida al crear la tabla o definida luego.

```
ALTER TABLE libros DROP PRIMARY KEY;
```

Si tenemos una clave primaria que posee el atributo "AUTO_INCREMENT", no se podrá eliminar, y nos aparecerá un mensaje de error avisandonos. Lo que debemos realizar primero es modificarlo y quitarle el atributo "AUTO_INCREMENT", para luego eliminar exitosamente la clave primaria.

Renombrar la tabla:

Podemos además cambiar el nombre de una tabla Usamos "ALTER TABLE" seguido del nombre actual, "RENAME" y el nuevo nombre.

Por ejemplo cambiaremos el nombre de la tabla "libros" por "ejemplares":

```
ALTER TABLE libros RENAME ejemplares;
```

También podemos cambiar el nombre a una tabla usando la siguiente sintaxis:

```
RENAME TABLE datos TO contactos;
```

Si queremos intercambiar el nombre de una tabla con otra, deberemos hacerlo de izquierda a derecha, utilizando un nombre temporal para el cambio. Por ejemplo intercambiamos los nombres de la tabla "amigos" y la tabla "contactos", utilizando "auxiliar":

```
RENAME TABLE amigos TO auxiliar, contactos TO amigos, auxiliar TO contactos;
```